

IoT Vitals — Wireless Pulse Oximetry Dashboard

A networked photoplethysmography system built on the ESP32

2026-09-20 · @Someone

Abstract

A wireless sensor node measures cardiac signals at a fingertip and streams them to a local server that stores the history and drives a live browser dashboard.

An MH-ET LIVE MAX30102 optical sensor illuminates tissue with red (660 nm) and infrared (880 nm) light and measures what returns. An ESP32 samples that signal at 100 Hz and derives heart rate, peripheral oxygen saturation (SpO_2), perfusion index and beat-to-beat intervals on-device, then publishes them over Wi-Fi using MQTT. A Node.js server ingests the stream, persists it to SQLite and pushes it to the browser over WebSocket. A 0.91-inch OLED at the node shows readings locally, so the device stays useful without a network.

The system is local-first: acquisition, processing, storage and presentation all run on the user's own network, with no cloud service in the path.

Objectives

1. Acquire a photoplethysmogram (PPG) from a fingertip using dual-wavelength reflectance.
2. Derive heart rate, SpO_2 and perfusion index on the microcontroller rather than on the server, so the node is autonomous.
3. Transmit readings wirelessly using a publish-subscribe protocol suited to constrained devices.
4. Persist measurements so that trends can be reviewed after a session ends.
5. Present live and historical data in a browser, updating without polling or page reloads.
6. Compute heart rate variability from beat-to-beat intervals, including a Poincaré plot.
7. Display readings on a local OLED so the node functions independently of the network.

System architecture

The system is a four-stage pipeline: optical acquisition, on-device signal processing, wireless transport, and server-side storage with browser presentation.

```
flowchart LR
```

```
  A[MAX30102<br/>optical sensor] -->|I2C 100 kHz| B[ESP32<br/>DSP + beat detection]
```

```
  B -->|I2C| C[SSD1306 OLED<br/>local readout]
```

```
  B -->|Wi-Fi / MQTT| D[Mosquitto<br/>broker]
```

```
  D -->|subscribe| E[Node.js server]
```

```
  E --> F[(SQLite<br/>history)]
```

```
  E -->|WebSocket| G[Browser<br/>dashboard]
```

Each stage is decoupled by a defined interface, so any one can be replaced without touching the others. The MQTT topic contract is the key boundary: the server does not know how readings were produced, and the firmware does not know what consumes them.

Stage	Runs on	Responsibility
Acquisition	MAX30102	18-bit red and IR samples at 100 Hz
Processing	ESP32	Bandpass filtering, beat detection, SpO ₂ , perfusion
Transport	Wi-Fi + MQTT	JSON payloads on per-device topics
Ingest and storage	Node.js + SQLite	Validation, persistence, HRV computation
Presentation	Browser	Live charts over WebSocket, history over REST

Hardware components

Component	Specification	Role
ESP32 DevKit V1	ESP32-D0WD-V3, dual-core Xtensa LX6 @ 240 MHz, 320 KB SRAM, 4 MB flash, Wi-Fi + BLE	Sampling, signal processing, MQTT client
MH-ET LIVE MAX30102	Integrated pulse oximeter, red 660 nm + IR 880 nm LEDs, 18-bit ADC, I ² C address 0x57	Optical acquisition of the PPG
SSD1306 OLED	0.91 inch, 128 × 32 monochrome, I ² C address 0x3C	Local readout at the node
CH343 USB-serial bridge	On-board, enumerates as a CDC-ACM device	Firmware upload and serial diagnostics

Component	Specification	Role
AMS1117-3.3 regulator	On-board, ~1 A	Derives the 3.3 V rail from USB 5 V

Why these parts

The **MAX30102** integrates both LEDs, a photodiode, the analog front end and an 18-bit ADC in one package, so the microcontroller receives digital samples over I²C rather than raw analog signals needing external amplification.

The **ESP32** was selected for integrated Wi-Fi and sufficient headroom for on-device filtering. Final firmware uses 16.2% of SRAM and 60.4% of flash, leaving ample margin.

Supplying the radio matters more than clock speed here: enabling the Wi-Fi PHY draws a step current of roughly 300–400 mA, so the 3.3 V rail must source that transient without sagging.

Circuit connections

Both peripherals share a single I²C bus in parallel. They are distinguished by address, not by wiring, so **SDA** and **SCL** carry two devices on the same two lines.

Signal	ESP32 DevKit V1	MAX30102	SSD1306 OLED
Power	3V3	VIN	VCC
Ground	GND	GND	GND
I ² C data	GPI021	SDA	SDA
I ² C clock	GPI022	SCL	SCL

flowchart TD

```

E[ESP32 DevKit V1] ---|3V3 / GND| P[Power rail]
E ---|GPI021 SDA| BUS[I2C bus]
E ---|GPI022 SCL| BUS
BUS --- M[MAX30102<br/>addr 0x57]
BUS --- O[SSD1306<br/>addr 0x3C]
P --- M
P --- O

```

Bus notes

- **Supply voltage is 3.3 V, not 5 V.** Both modules carry on-board regulation and level shifting, and the ESP32's GPIO pins are not 5 V tolerant.
- **Pull-up resistors are already present** on the MAX30102 breakout, so no external pull-ups are required.
- **The bus runs at 100 kHz** rather than the 400 kHz fast mode. Two devices plus jumper leads add capacitance, and the lower rate gives margin against signal degradation.
- **Addresses do not collide:** 0x57 for the sensor, 0x3C for the display. Firmware scans the bus at boot and reports what answers, which confirms wiring before any measurement is attempted.
- Unused MAX30102 pins (`INT` , `RD` , `IRD`) are left unconnected.

Firmware design

The firmware is written in C++ against the Arduino framework and built with PlatformIO. All signal processing happens on the ESP32; the server receives derived values, not raw samples.

Sensor configuration

Parameter	Value	Reason
LED mode	Red + IR	Two wavelengths are required for SpO ₂
Sample rate	400 Hz, averaged × 4	Yields a 100 Hz effective stream
Pulse width	411 μs	Enables full 18-bit ADC resolution
LED amplitude	0x3C	Balances signal strength against saturation
ADC range	16384	A fingertip pegs the narrower 4096 range at full scale, clipping the entire pulsatile component

Processing chain

```

flowchart LR
    A[FIFO<br/>red + IR] --> B[Finger<br/>detect]
    B --> C[Bandpass<br/>0.16-4 Hz]
    C --> D[Beat<br/>detect]
    D --> E[AC / DC<br/>ratio]
    E --> F[Median IBI<br/>to BPM]
    F --> G[SpO2 +<br/>perfusion]
  
```

Finger detection. A fingertip raises the IR DC level from roughly 250 counts to tens of thousands. Crossing a 12,000-count threshold marks contact; below it, every derived value is cleared rather than left stale.

Cardiac bandpass. A fast exponential moving average ($\tau \approx 0.25$ s) minus a slow one ($\tau \approx 1$ s) isolates the 0.16–4 Hz band a pulse occupies. Measuring amplitude on the unfiltered signal instead would sum all out-of-band noise into the result and bias SpO₂ low.

Beat detection. Peaks are found on the infrared channel. Intervals outside 300–2000 ms (30–200 bpm) are rejected as artefacts, and the reported rate is derived from the **median** of the last eight intervals — a single missed beat doubles one interval, which would drag a mean down by roughly 10 bpm while leaving a median unmoved.

Oxygen saturation. Oxygenated and deoxygenated haemoglobin absorb 660 nm and 880 nm light differently, so the ratio of pulsatile to steady absorption at the two wavelengths tracks saturation:

$$R = \frac{AC_{\text{red}}}{DC_{\text{red}}} \frac{AC_{\text{IR}}}{DC_{\text{IR}}}$$

$$SpO_2 = -45.060 \cdot R^2 + 30.354 \cdot R + 94.845$$

AC is the RMS of the bandpassed signal over a 4-second window; DC is the window mean.

Quality gating. Three conditions suppress the SpO₂ output rather than publishing a plausible-looking but meaningless number: ADC saturation, perfusion below 0.15%, and a window whose pulse amplitude jumps more than 60% against its running average. That last case is motion — it modulates both wavelengths almost equally, driving R toward 1, which the polynomial maps to a confident but false reading near 80%.

Communication protocol

MQTT was chosen over HTTP because it is designed for constrained devices: a persistent connection avoids per-message handshakes, and publish-subscribe lets any number of consumers attach without the node knowing about them.

All topics are namespaced per device as `vitals/<device-id>/<channel>`.

Topic	Rate	Payload
<code>.../vitals</code>	1 Hz	<code>{t, bpm, spo2, pi, finger, ir, rdc, rssi, up, sensor, sat}</code>
<code>.../ppg</code>	2 Hz	<code>{t, fs, ppg: [25 int16]}</code> — baseline-removed IR at 50 Hz
<code>.../ibi</code>	per beat	<code>{t, ibi: [ms, ...]}</code> — intervals between accepted beats
<code>.../status</code>	on change	<code>online / offline</code> , retained

Design decisions

Last will and testament. The node registers `offline` as its MQTT will at connection time. If it loses power or drops off the network, the broker publishes that message on its behalf, so the dashboard shows a disconnected device instead of freezing on its final reading.

Retained status. The status topic is retained, so a dashboard opened later immediately learns the device state without waiting for the next change.

Waveform batching. Sending 50 individual samples per second would be wasteful. Samples are batched 25 at a time and published twice a second, cutting message overhead by a factor of 25.

Beats are published individually, not averaged. Heart rate variability is defined by the spacing between beats, so averaging before transmission would destroy the quantity being measured.

Device time is not trusted. The `t` field is milliseconds since boot, not wall-clock. The server timestamps everything on receipt and keeps `up` only as an uptime readout.

Backend server and storage

The server is a Node.js application with only two runtime dependencies, `mqtt` and `ws`. History uses Node's built-in `node:sqlite` module, so no native database driver has to be compiled.

Responsibilities

1. Subscribe to `vitals/+/+` and validate incoming JSON.
2. Stamp every reading with server time.

3. Persist measurements and individual heartbeats to SQLite.
4. Maintain an in-memory window of recent waveform samples and beat intervals.
5. Compute HRV metrics on each new beat.
6. Broadcast updates to connected browsers over WebSocket.
7. Serve the dashboard and a small REST API.

Database schema

```
CREATE TABLE readings (
  id      INTEGER PRIMARY KEY,
  device  TEXT      NOT NULL,
  ts      INTEGER NOT NULL,    -- server epoch ms
  bpm     REAL,
  spo2    REAL,
  pi      REAL,
  rssi    INTEGER
);

CREATE TABLE beats (
  id      INTEGER PRIMARY KEY,
  device  TEXT      NOT NULL,
  ts      INTEGER NOT NULL,
  rr      INTEGER NOT NULL    -- interval to previous beat, ms
);
```

Write-ahead logging is enabled so reads never block the ingest path. Readings are stored **only while a finger is detected**, so the charts show real measurement sessions rather than long runs of zeros, and gaps between sessions break the plotted line instead of being drawn through.

REST API

Route	Returns
GET /api/health	Broker connection state, device and client counts
GET /api/devices	Live devices and everything ever recorded
GET /api/history?device=&range=	Bucketed averages over 5m to 24h
GET /api/hrv?device=&minutes=	HRV metrics and Poincaré pairs
WS /ws	Snapshot on connect, then live frames

History queries downsample server-side by grouping into time buckets, so a 24-hour range returns a few hundred points rather than 86,000 rows.

Web dashboard

The dashboard is served as plain ES modules with no build step or framework, keeping the browser code inspectable and the toolchain minimal. It is organised as three views, switched from a sidebar and routed by URL hash so views are linkable.

View	Contents
Live	Heart-rate hero figure, signal-quality meter, SpO ₂ and perfusion, PPG waveform
Variability	HRV metrics, Poincaré plot, beat-interval tachogram
History	Range selector, heart-rate and SpO ₂ trends, table view

Visualisation decisions

Separate charts, never a dual axis. Heart rate and SpO₂ occupy very different ranges. Plotting them against two y-scales would invent an apparent correlation, since the alignment of the two scales is arbitrary.

Gaps are breaks, not slopes. When no finger is present nothing is recorded, and the chart breaks the line rather than interpolating across the gap.

The waveform is gated on contact. Without a finger the trace is dark-current noise a few counts wide; autoscaling that would fill the frame with something resembling a signal, so the panel shows a prompt instead.

Numbers are withheld when unsupported. Tiles show a dash rather than a zero when the signal does not justify a reading, distinguishing "not measured" from "measured as zero".

Colour accessibility

Series colours were validated programmatically rather than chosen by eye, against five criteria: lightness band, chroma floor, colour-vision-deficiency separation, normal-vision separation, and contrast against the surface.

The selected pair separates well beyond the target: CVD ΔE 42.3 in dark mode against a target of 8. An earlier candidate pairing was rejected because it measured ΔE 1.6 under simulated deuteranopia — effectively identical to a colourblind reader, despite looking clearly distinct to normal vision.

Where a hue falls below the 3:1 contrast floor, the relief rule applies: every chart carries a visible direct label on its latest value, and a table view exposes the same numbers with no colour at all.

Heart rate variability analysis

Heart rate variability measures the spacing between individual beats rather than their average rate. Two people at the same pulse can differ substantially, because HRV reflects autonomic balance rather than cardiac output.

Metric	Definition
RMSSD	Root mean square of successive interval differences; the standard short-term measure
SDNN	Standard deviation of the intervals; longer-term variability
pNN50	Share of consecutive pairs differing by more than 50 ms
SD1	Poincaré spread perpendicular to the line of identity (equals $RMSSD / \sqrt{2}$)
SD2	Poincaré spread along the line of identity

$$RMSSD = \sqrt{\frac{1}{N-1} \sum_{i=1}^{N-1} \left(RR_{i+1} - RR_i \right)^2}$$

Artefact rejection

RMSSD is a difference measure, which makes it acutely sensitive to exactly the errors an optical sensor produces: a missed beat doubles an interval, a double-counted one halves it. Intervals deviating more than 20% from the running median are rejected before any metric is computed.

The count of rejected intervals is displayed alongside the results, so a noisy recording is visibly noisy rather than silently averaged. No metric is reported below 20 clean beats; until then the interface states how many more it needs.

Poincaré plot

Each interval is plotted against the one that follows it. The shape carries the reading: a tight cluster along the diagonal indicates low variability, a wide cloud indicates high, and ectopic beats appear as distinct satellites off the diagonal that no single summary number would reveal.

Both axes share one scale so the geometry is undistorted, and an ellipse with semi-axes SD1 and SD2, rotated onto the line of identity, is overlaid to make the two components legible.

Results

The assembled system produced the following measurements from a fingertip on the completed hardware.

Quantity	Measured	Notes
Heart rate	72–89 bpm resting	Stable to within a few bpm over a held window
SpO ₂	98.0% (range 95.3–99.7)	Physiologically plausible for a healthy subject
Perfusion index	0.83–4.79%	Above the 0.15% floor required for a valid ratio
IR DC level	~65,000 counts	About 25% of full scale; good headroom, no clipping
Dark baseline	245–590 counts	Correct no-finger value
RSSI	–59 to –66 dBm	Reliable link throughout

System performance

Measure	Value
Firmware SRAM usage	16.2% of 320 KB
Firmware flash usage	60.4% of 1.31 MB
Effective sample rate	100 Hz
Vitals publish rate	1 Hz
Waveform publish rate	2 Hz (25 samples per message)

Measure	Value
Server dependencies	2 (<code>mqtt</code> , <code>ws</code>)

The I²C scan at boot consistently identifies both peripherals — `0x3C` for the display and `0x57` for the sensor, which reports part ID `0x15` revision `0x03` — confirming the bus and wiring before measurement begins.

Heart rate proved the most robust output. SpO₂ required the bandpass and motion gating described above before it became trustworthy. HRV is the most demanding of all, since it depends on every individual beat being detected correctly.

Software stack and operation

Layer	Technology
Firmware	C++ / Arduino framework, built with PlatformIO
Sensor driver	SparkFun MAX3010x library
Display driver	Adafruit SSD1306 + GFX
MQTT client (device)	PubSubClient
Broker	Eclipse Mosquitto 2, in Docker
Server	Node.js 24 (<code>mqtt</code> , <code>ws</code> , built-in <code>node:sqlite</code>)
Dashboard	Plain ES modules, Canvas 2D, no build step

Running the system

A single shell script manages the whole stack:

```
./start.sh          # broker + server, opens the dashboard
./start.sh status   # what is running, and what each node reports
./start.sh stop     # stops both
./start.sh flash    # build and upload firmware
./start.sh monitor  # serial monitor
```

The script is idempotent — running it twice reports what is already up rather than starting duplicates. It installs server dependencies on first use and prints the broker's LAN address, which is the value the firmware needs as `MQTT_HOST`.

Configuration

Network credentials live in `firmware/include/secrets.h`, which is excluded from version control. A template at `secrets.example.h` documents the required fields: Wi-Fi SSID and password, broker address and port, and a device identifier that becomes the MQTT topic prefix.

The broker must be reachable **from the device's network**, so `MQTT_HOST` is the LAN address of the machine running Mosquitto, not `localhost`.

Limitations and future work

Limitations

Not a medical device. The MAX30102 is a hobbyist optical sensor and readings are uncalibrated. The SpO_2 polynomial is an empirical curve for this sensor family, not a calibration traceable to a reference oximeter. The system is for learning about photoplethysmography, not for health decisions.

Motion sensitivity. Movement changes the optical path rather than blood oxygenation and modulates both wavelengths nearly equally. The firmware detects and rejects those windows, but the result is fewer valid readings rather than correct readings during motion.

Contact dependence. Perfusion index varies with pressure and skin temperature. Pressing too firmly occludes the capillaries being measured, so light steady contact produces better results than a hard grip.

HRV needs clean beats. RMSSD is a difference measure, so undetected artefacts inflate it. A recording with many rejected intervals should be treated as indicative only.

Single-node scope. The architecture supports multiple devices — topics are namespaced and the dashboard has a device picker — but only one node was built and tested.

Future work

1. **Calibrate SpO_2** against a reference oximeter by collecting paired measurements and fitting coefficients for this specific sensor, replacing the generic polynomial.
2. **Derive respiration rate** from the PPG. Breathing modulates pulse amplitude, interval timing and baseline; bandpassing the interval series at 0.1–0.5 Hz would yield a second vital sign from the same sensor.
3. **Battery operation** with deep sleep between measurement sessions, making the node genuinely portable.

4. **Multiple nodes** for simultaneous subjects, which the topic namespacing already accommodates.
5. **Guided breathing biofeedback**, pacing respiration at roughly six breaths per minute on the OLED while displaying HRV in real time.
6. **Session recording and export** to CSV for offline analysis.